CSE-353: System Analysis and Design

Section-B — Model Solutions (2019–2024)

Source focus used:

- *The Unified Modeling Language User Guide (2nd Ed.)*, Chapter 1–2
- *Kendall & Kendall: Systems Analysis and Design*, Chapter 3, 6, 14, 15, 16

> Note: Some original questions mention tables/figures/scenarios that are not fully included in the prompt. For those, the solution method and correct template are provided so you can directly plug in the exam data.

2024 (Level-3, Term-I)

Q5(a) Building blocks of UML with diagrams

UML has three core building blocks:

1. **Things** (structural, behavioral, grouping, annotational)
2. **Relationships** (dependency, association, generalization, realization)
3. **Diagrams** (use case, class, sequence, activity, state, component, deployment, etc.)

Simple class-diagram view:

```
Student ----enrolls----> Course
   ^                        ^
   |                        |
inherits                associated with
GraduateStudent      CourseOffering
```

Q5(b) Use case diagram (course registration)

Actors: **Student, Advisor, Authentication Service, Payment Gateway (if fee), Registrar DB**

Main use cases: Login, View Dashboard, Search Course, Add to Cart, Submit Registration, Waitlist, Drop Course, Request Approval, Advisor Approves/Rejects.

Use-case sketch:

```
Student --> (Login) --> (View Dashboard)
Student --> (Search Course)
Student --> (Add to Cart)
Student --> (Submit Registration)
(Submit Registration) --> (Check Seat Availability)
(Check Seat Availability) --> (Enroll) / (Put in Waitlist)
Student --> (Drop Course)
Student --> (Request Advisor Approval)
Advisor --> (Approve/Reject Request)
```

Q5(c) How UML helps OO design

- Makes requirements and design **visual and unambiguous**.
- Supports **abstraction** (class, object, interaction views).

- Enables early validation through use case, sequence, and activity models.
- Improves communication among users, analysts, developers, testers.
- Provides traceability from requirements → design → implementation.

Q6(a) Six Sigma and analyst relevance

Six Sigma is a data-driven quality methodology to reduce variation and defects (target ~3.4 defects per million opportunities).

Analysts must understand:

- **DMAIC**: Define, Measure, Analyze, Improve, Control.
- Metrics-based process improvement.
- Root-cause analysis and continuous improvement culture.
- Alignment between business goals and measurable system performance.

Q6(b) Business graphics and report format guidelines

Business graphics: visual presentation of business information (charts, dashboards, trend graphs, heat maps, KPI panels).

Good report format:

- Clear objective and audience orientation.
- Correct chart type for data type.
- Readable layout, labels, legends, scales.
- Highlight exceptions and actionable insights.
- Consistent typography/colors.
- Minimal clutter; maximize signal-to-noise.

Q6(c) Modulus-11 check: code 449632

Assume standard weighted Mod-11 generation for first 5 digits with weights 6,5,4,3,2:

- Weighted sum = $4×6 + 4×5 + 9×4 + 6×3 + 3×2 = 104$
- Remainder = $104 \mod 11 = 5$
- Check digit = $11-5 = 6$ (or 0 if result 11)

Expected full code: **449636**, not 449632.

Therefore, the entered code is incorrect.

Q6(d) Maintaining data integrity in database

- Entity integrity (PK not null, unique).
- Referential integrity (FK constraints, cascading rules).
- Domain integrity (type/range/check constraints).
- Transaction integrity (ACID, commit/rollback).
- Concurrency control (locking/versioning).
- Validation rules at UI + service + DB levels.

- Backup, recovery, audit trails.

Q7(a) Prototyping as SDLC alternative + guidelines

Prototyping can replace rigid linear flow when requirements are unclear:

- Build quick working model.
- Collect user feedback early.
- Iterate until requirements stabilize.

Guidelines:

- Time-box iterations.
- Start with high-risk/high-value features.
- Keep scope controlled.
- Maintain user involvement.
- Document assumptions and changes.
- Convert approved prototype into engineered production design.

Q7(b) Fishbone causes for food delivery app failure

Main bones:

- **People:** skill gaps, weak communication, poor training.
- **Process:** unclear requirements, weak QA, bad sprint planning.
- **Technology:** unstable APIs, scaling failure, weak security.
- **Data:** wrong menus/pricing/location data.
- **External:** payment gateway downtime, map API failure, legal issues.
- **Operations:** rider shortage, dispatch latency, poor incident response.

Q7(c) HCI “fit” and well-being

Fit = alignment among **human capabilities**, **computer/interface design**, and **task demands**.

Good fit yields:

- Faster completion time.
- Lower cognitive load and error rate.
- Better satisfaction, lower stress and fatigue.
- Sustainable long-term performance.

Q7(d) Structured walkthrough

A structured walkthrough is a formal peer review of analysis/design/code artifacts to detect defects early.

Participants:

- Author/presenter
- Moderator
- Peers (analyst/developer/tester)

- User representative/domain expert
- Recorder/scribe

Q8(a) Steel grading decision table (EEDT + correctness)

Because exact thresholds are not fully included, use this EEDT structure:

Conditions:

- C1: Carbon content (Low/Acceptable/High)
- C2: Hardness (Pass/Fail)
- C3: Tensile strength (Pass/Fail)

Actions:

- A1: Grade A
- A2: Grade B
- A3: Reject

Rules should be **mutually exclusive and collectively exhaustive**.

Correctness checks:

1. No contradictory actions in one rule.
2. Every possible condition combination covered.
3. No duplicate/redundant rule columns.
4. “Else” rules only where justified.

Q8(b) Physical DFD usefulness + external entity vs process

Physical DFD helps by showing **who does what, where, and with which media/devices** (forms, files, terminals, departments).

Difference:

- **External Entity:** outside system boundary; source/sink of data.
- **Process:** transforms input data into output within system.

Q8(c) Web fill-in forms pros/cons + good coding system requirements

Pros:

- Fast capture, validation, instant feedback, lower paper handling cost.

Cons:

- Accessibility/usability issues, network dependence, security/privacy risks.

Good coding system requirements:

- Unique, expandable, meaningful where needed.
- Standardized format and fixed length where practical.
- Easy to input/read; avoids ambiguous characters.

- Supports validation (check digit).
- Supports sorting/classification and future growth.

2023 (Level-III, Term-I)

Q5(a) Why UML?

- Standard visual language for complex OO systems.
- Reduces ambiguity and improves design quality.

Example: class + sequence diagrams for online booking clearly show structure and behavior.

Q5(b) When users need interface feedback

- After user action submission.
- During long processing.
- On validation errors.
- On successful completion.
- On system status changes (saved/synced/disconnected).

Q5(c) Why efficient data capture matters + how

Quality data prevents wrong decisions and costly rework.

Efficient capture via:

- Input masks, defaults, dropdowns, scanning/OCR.
- Real-time validation.
- Minimize re-entry, auto-fill known data.

Q5(d) Risks adopting new information systems

- User resistance, training gaps.
- Data migration errors.
- Cost/time overruns.
- Security vulnerabilities.
- Process disruption and productivity dip during transition.

Q6(a) Car reservation use case scenario + diagram

Actors: Traveler, Reservation Agent, Payment Gateway, Vehicle Inventory Service.

Use cases: Search Car, View Fare, Select Car, Enter Traveler Info, Pay, Confirm Booking, Modify/Cancel.

```
Traveler -> (Search Car) -> (Select Car) -> (Pay) -> (Get Confirmation)
Traveler -> (Modify/Cancel Booking)
Payment Gateway <-> (Pay)
Inventory Service <-> (Search Car)
```

Q6(b) “Prototype has some, not all essential features”

A prototype is intentionally partial:

- Includes critical flows for feedback.
- Omits full optimization/security/performance/hardening.
- Used for learning and requirement refinement, not final production quality.

Q6(c) PERT over Gantt

Advantages:

- Shows task dependencies explicitly.
- Identifies critical path and slack.
- Supports uncertainty (optimistic/most likely/pessimistic estimates).
- Better for risk-sensitive schedule planning.

Q7(a) UI as first critical step for user needs

Users react better to screens/workflows than abstract specifications.

Early UI mockups reveal hidden requirements, usability issues, and terminology mismatch.

Q7(b) Network/slack/critical path with provided table

Use CPM procedure:

1. Build AON network from predecessor table.

2. Forward pass: compute ES/EF.

3. Backward pass: compute LS/LF.

4. Slack = LS–ES (or LF–EF).

5. Critical path: activities with slack = 0.

6. If activity I duration becomes 10 weeks, recompute path containing I; project duration changes only if that path becomes/extends critical.

Q7(c) Better coding for ticket holders

Suggested code: `YY-SEASON-ZONE-ROW-SEAT-CHECK` (e.g., `24-WIN-A-12-08-7`)

Benefits:

- Encodes context and location.
- Reduces mix-ups by structure + check digit.
- Easier verification and error detection.

Q8(a) User story: written or spoken?

Primary artifact should be **written briefly**, then elaborated through conversation.

Example: “As a student, I want to add/drop courses so that I can optimize my semester load.”

Q8(b) Parallel conversion problem + reassurance strategy

Problems:

- Double operating cost.
- Data inconsistency risk between systems.
- User confusion and delayed full adoption.

Reassurance strategy:

- Use staged exit criteria (accuracy, uptime, reconciliation thresholds).
- Pilot subset first, then formal cutover date with rollback plan.

Q8(c) Check digit for 238933 using 1-3-1 and mod 11

Apply repeating weights 1,3,1,3,1,3:

- Sum = $2×1 + 3×3 + 8×1 + 9×3 + 3×1 + 3×3 = 58$
- $58 \mod 11 = 3$
- Check digit = $11-3 = 8$

Final code: **2389338**.

Q8(d) TQM in analysis and design

TQM means continuous quality focus:

- Customer-driven requirements.
- Prevention over correction.
- Process standardization and measurement.
- Cross-functional teamwork and continual improvement cycles.

2022 (Level-3, Term-I)

Q5(a) Attributes of coding system + modulus-11 technique

Attributes:

- Uniqueness, expandability, simplicity, standardization, validation support, usability.

Modulus-11:

- Weighted digits summed; remainder mod 11 determines check digit.

Condition:

- Check digit chosen so final weighted sum satisfies divisibility rule under selected scheme.

Q5(b) Error-reducing principles in input design

- Capture at source.
- Validate immediately.
- Use constrained controls (dropdown/radio/date picker).
- Use clear labels and formatting.

- Keep forms short and logically grouped.
- Confirm critical entries.

Q5(c) Linked decision table for promotion rules

When many conditions/actions exist, split into linked tables:

- Table 1: eligibility screening.
- Table 2: scoring/point outcome.
- Table 3: final action (promote/defer/reject).

Ensure links are deterministic and complete.

Q6(a) Data dictionary + example

Data dictionary = central metadata repository of data elements, structures, meanings, rules.

Example entry:

- Name: `Student_ID`
- Type: `CHAR(10)`
- Format: `YYYYNNNNNN`
- Constraints: unique, not null
- Description: unique student identifier

Q6(b) Effective capture and quality

Needed to reduce garbage-in/garbage-out effects.

Efficient methods: automation, real-time checks, standards, reuse of master data, user training.

Q6(c) Why UML with example

UML offers standardized visualization and better team communication.

Example: use case + class diagram for library reservation module.

Q6(d) Functional dependency; 4NF and 5NF

FD: $X \rightarrow Y$ means X uniquely determines Y.

4NF needed when removing non-trivial multivalued dependencies.

5NF needed when decomposing relations to remove join dependencies and avoid spurious tuples.

Q7(a) Role of user in agile modeling

- Prioritizes stories.
- Validates increments.
- Gives continuous feedback.
- Clarifies domain rules.
- Co-owns acceptance criteria.

Q7(b) Prototype delay scenario

i) Rapid partial delivery is important for early risk reduction and requirement correction.

ii) Control by time-boxing, scope freeze per iteration, clear acceptance criteria, frequent demos.

Q7(c) Client/server advantages and disadvantages

Advantages: scalability, centralized control, resource sharing, easier updates on server side.

Disadvantages: network dependency, server bottlenecks, security concentration, higher administration needs.

Q8(a) Why choose object-oriented approach

- Natural mapping to real-world entities.
- Reuse through inheritance/composition.
- Better maintainability and extensibility.
- Supports iterative development and design patterns.

Q8(b) Library system use case diagram (text form)

Actors: Patron, Librarian, Payment Service, Notification Service.

Use cases: Login, Search Book, Reserve Book, Report Lost Book, Pay Fine, Receive Notification.

```
Patron -> (Login)
Patron -> (Search Book)
Patron -> (Reserve Book)
Patron -> (Report Lost Book) -> (Pay Fine)
Payment Service <-> (Pay Fine)
Notification Service <-> (Reserve Book)
```

Q8(c) Sustainability justification for Q8(b) system

- Modular services and clean interfaces.
- Scalable infrastructure.
- Security/privacy by design.
- Maintainable data model and auditability.
- Operational KPIs and feedback loops for continuous improvement.

2021 (Level-3, Term-II)

Q5(a) Improve navigation ease and stickiness in e-commerce

- Consistent IA and menu hierarchy.
- Fast search, filters, breadcrumbs.
- Personalized recommendations.
- Quick checkout and saved preferences.
- Trust signals (reviews, return policy, secure payment).
- Mobile-first responsive design.

Q5(b) Network/slack/critical path and activity E impact

Use CPM method (forward/backward pass, slack).

Saving time on E helps only if E lies on the critical path (slack = 0); otherwise total duration remains unchanged.

Q5(c) Critique and redesign code `LO2002Z621289`

i) Likely issues: mixed letter-number ambiguity, no separators, hard to validate visually.

ii) Redesign with grouped segments + check digit, e.g., `LO-2002-Z6-21289-4`.

iii) Eliminates transposition/single-key errors and reading mistakes.

Q6(a) Efficient data capture and quality

High-quality inputs reduce downstream correction and decision errors.

Apply field constraints, defaults, lookup tables, scanning, and immediate exception prompts.

Q6(b) Data dictionary + Structured English vs decision table/tree

- Structured English: process logic in constrained natural language.
- Decision table: compact condition-action matrix.
- Decision tree: branching visual logic.

Yes, one task can be represented in all three forms; choose based on complexity and audience.

Q6(c) LEDT, MEDT, correctness (Physics/Math rules)

Procedure:

1. Define conditions/actions exactly from rule text.
2. Build **LEDT** by listing all combinations.
3. Collapse don’t-care conditions to build **MEDT**.
4. Verify no missing/contradictory/duplicate rules.

Q6(d) Requirements of good coding system

- Unambiguous, concise, standardized, expandable.
- Easy entry and machine processing.
- Includes validation mechanism.

Q7(a) Functional dependency, 4NF, 5NF

Same core answer as 2022 Q6(d): FD defines determinant relation; 4NF handles MVD anomalies; 5NF handles join dependency anomalies.

Q7(b) How analysis helps design

Analysis discovers what system must do; design determines how to do it.

Strong analysis reduces rework, improves fit, controls scope/cost, and yields traceable architecture decisions.

Q7(c) Flo scenario and prototyping

i) Prototyping helps freeze changing requirements through visible increments and frequent validation.

ii) Users articulate needs better when they interact with concrete screens/workflows rather than abstract documents.

Q8(a) Pros/cons of web fill-in forms

Pros: speed, automatic checks, paperless process, immediate submission.

Cons: usability barriers, browser/device issues, privacy/security concerns, internet dependence.

Q8(b) E-commerce vs traditional PM

- Faster release cycles and continuous deployment.
- Higher uncertainty and rapid market-driven reprioritization.
- Stronger dependency on UX, analytics, cybersecurity, and uptime.
- Heavier integration with payment/logistics platforms.

Q8(c) FOLKLORE documentation vs traditional

FOLKLORE captures tacit user knowledge via narratives, stories, observed practice.

Traditional documentation is formal, structured, and process-centered.

FOLKLORE is richer contextually; traditional is more standardized and auditable.

Q8(d) Agile modeling values and principles

Values: communication, simplicity, feedback, courage, humility.

Principles: model with purpose, travel light, embrace change, rapid feedback, stakeholder collaboration.

2020 (Level-3, Term-II)

Q5(a) Object-oriented modeling, properties, and UML as OO language

OOM represents systems as interacting objects/classes.

Properties: abstraction, encapsulation, inheritance, polymorphism, message passing.

UML is OO modeling language because it directly models classes, objects, interactions, and OO relationships.

Q5(b) When to use SDLC, Agile, OO approaches

- **SDLC (plan-driven):** stable requirements, regulatory control.
- **Agile:** evolving requirements, rapid delivery need.
- **OO approach:** complex domain with reusable components and long-term maintainability focus.

Q5(c) Six Sigma for analysts

Analysts use Six Sigma thinking to quantify defects, identify process variation, and improve system quality through DMAIC and metrics.

Q5(d) Report format points

- Clear objective/title, meaningful grouping, concise summaries.
- Visual hierarchy, readable typography, consistent formats.
- Exception highlighting and decision-oriented conclusions.

Q6(a) Purpose of data validation + modulus-11 for 96432 + algorithms

Purpose: prevent invalid data entry, protect integrity and processing correctness.

For 96432 (weights 6,5,4,3,2):

- Sum = $9×6 + 6×5 + 4×4 + 3×3 + 2×2 = 113$
- Remainder $= 113 \mod 11 = 3$
- Check digit $= 11-3 = 8$

Code: **964328**

Generation algorithm:

1. Multiply each base digit by defined weight.
2. Sum products.
3. Compute remainder mod 11.
4. Check digit = (11 – remainder) mod 11.
5. Append check digit.

Error detection algorithm:

1. Compute weighted sum including check digit.
2. If divisibility condition fails, reject code as invalid.

Q6(b) Pizza ordering use case diagram

Actors: Customer, Payment Gateway, Delivery System.

Use cases: Login, Search Pizza, Add to Cart, Provide Address, Pay, Place Order, Track Delivery.

```
Customer -> (Login)
Customer -> (Search Pizza) -> (Add to Cart)
Customer -> (Provide Address) -> (Pay) -> (Place Order)
Delivery System <-> (Track Delivery)
Payment Gateway <-> (Pay)
```

Q6(c) Prototyping as SDLC alternative

Use iterative prototype cycles to converge on requirements quickly; follow scope control, time-boxing, user review, and refinement discipline.

Q6(d) HCI fit and effects

Fit means matching user capability, interface behavior, and task complexity.

Better fit improves speed, accuracy, comfort, and trust; poor fit increases fatigue, errors, and abandonment.

Q7(a) CASE tools

CASE = Computer-Aided Software Engineering tools supporting analysis, design, coding, testing, and documentation.

Importance: productivity, standardization, traceability, quality.

Examples: Enterprise Architect, Visual Paradigm, Rational Rose, StarUML.

Q7(b) Purpose of data validation program

To block invalid, incomplete, inconsistent data at entry and preserve data quality for reliable outputs and decisions.

Q7(c) Business graphics + agile modeling values/principles

Business graphics communicate trends/comparisons quickly (bar, line, pie, dashboard).

Agile modeling values/principles: purpose-driven minimal models, fast feedback, collaboration, adaptability.

Q7(d) Requirements of good coding system

Unique, simple, expandable, standardized, machine-processable, error-detectable, user-friendly.

Q8(a) Network/slack/critical path from table

Apply CPM exactly as described earlier: build network, compute ES/EF/LS/LF, slack, and identify zero-slack path as critical path.

Q8(b) RAD model and phases

RAD = Rapid Application Development emphasizing fast prototyping and component reuse.

Phases (typical):

1. Requirements planning
2. User design (workshops/prototyping)
3. Construction
4. Cutover (testing, training, deployment)

Q8(c) Efficient capture for data quality

Use source capture, validation, standard codes, minimal manual entry, and immediate exception handling.

Q8(d) When interface feedback is needed

Needed on command acknowledgment, progress indication, validation errors, warnings, and completion/next-step guidance.

2019 (Level-3, Term-II)

Q5(a) Six Sigma

Six Sigma is a disciplined quality-improvement approach to reduce process defects and variation using statistical thinking (DMAIC).

Analysts apply it to requirement quality, process performance, and continuous system improvement.

Q5(b) Short notes

- **Testing:** verification/validation to ensure requirement compliance and defect reduction.
- **Cloud computing:** on-demand scalable computing resources (IaaS/PaaS/SaaS).
- **Maintenance & auditing:** post-deployment correction/adaptation/perfection and formal checks for compliance, controls, and reliability.

Q5(c) Modulus-11 for 48793 + algorithms

Using weights 6,5,4,3,2:

- Sum = $4×6 + 8×5 + 7×4 + 9×3 + 3×2 = 125$
- $125 \mod 11 = 4$
- Check digit = $11-4 = 7$

Code: **487937**

Generation and detection algorithms are same as in 2020 Q6(a).

Q5(d) Good coding system requirements with example

Requirements: unique, concise, expandable, meaningful if needed, standard, easy to validate/input.

Example: `DEP-YY-EMPID-C` (department-year-employee-check digit).

Q6(a) Prototyping as SDLC alternative + guidelines

Use iterative prototype cycles for requirement clarification; apply time-boxing, scope control, prioritized features, frequent user validation.

Q6(b) RAD and phases

RAD focuses on rapid delivery through prototyping and reusable components; phases: planning, user design, construction, cutover.

Q6(c) Risks adopting new information systems

- Resistance to change
- Operational disruption
- Data migration/integration failures
- Security and privacy issues
- Cost overrun and schedule slip
- Vendor dependency

Q6(d) Agile modeling values/principles

Collaborative, adaptive, lightweight modeling with continuous feedback and purpose-driven artifacts.

Q7(a) Business graphics + offline vs online output

Business graphics present business metrics visually.

Offline output: printed/static reports, less interactive.

Online output: live dashboards, drill-down, real-time updates.

Q7(b) Functional dependency, 4NF, 5NF

FD defines determinant-based consistency; 4NF removes multivalued dependency anomalies; 5NF addresses join dependency decomposition correctness.

Q7(c) Airport check-in/security use case diagram

Actors: Passenger, Check-in Agent, Security Officer.

Use cases: Individual Check-in, Group Check-in, Issue Boarding Pass, Security Screening.

```
Passenger -> (Individual Check-in)
Passenger -> (Group Check-in)
(Individual Check-in) -> (Issue Boarding Pass)
(Group Check-in) -> (Issue Boarding Pass)
Passenger -> (Security Screening)
Security Officer -> (Security Screening)
```

Q8(a) Object selection and CRC cards

Object selection:

- Identify domain nouns (candidate classes).
- Keep classes with clear responsibility and behavior.
- Define relationships and collaborations.

CRC (Class-Responsibility-Collaborator) card format:

- Class: `Reservation`
- Responsibilities: create, modify, cancel reservation
- Collaborators: `Customer`, `Payment`, `Inventory`

Q8(b) Use case diagram from table (Use Case vs Actor)

Method:

1. List actors from table.

2. Map each use case to responsible actor(s).

3. Add include/extend where reuse/optional behavior exists.

4. Validate completeness against all actor goals.

Q8(c) Why UML with example

UML gives standard, reusable visual notation that supports design quality and communication.

Example: use case for “Course Registration” + class diagram (`Student`, `Course`, `Enrollment`) + sequence for add/drop flow.

Quick Reference: Decision Table Correctness Checklist

1. **Completeness:** all condition combinations covered.
2. **Consistency:** no rule conflict.
3. **Non-redundancy:** no duplicate columns.
4. **Feasibility:** each rule can occur in real life.
5. **Action integrity:** each rule has valid action set.

Quick Reference: Mod-11 (used in this document)

- Weights for 5-digit base examples: **6,5,4,3,2**
- Check digit: \((11 - (weighted_sum \mod 11)) \mod 11\)